Incremento, switch e cicli di ripetizione

Lezione 12 — 1IOT, SST-ICT-01 — Assegnazione composta, x++ e ++x, il costrutto switch, i cicli for, while e do while

19/10/2026

1 Gli argomenti di oggi

Due argomenti: l'**incremento** di una variabile e i **cicli di ripetizione**. In mezzo, per completare i **costrutti decisionali** visti con l'`if`, c'è anche il costrutto **switch**. Alla fine della lezione, alcuni esempi provati dal vivo su **onlinegdb**.

2 L'incremento

Incrementare una variabile vuol dire **aggiungerle una certa quantità**. Partiamo da un esempio:

```
int x;
x = 6;
x = x + 3;
```

Quanto vale `x` alla fine? **9**. Il motivo è il funzionamento dell'**uguale**, detto ormai molte volte: l'uguale **prende il valore** che c'è a destra e lo mette nella variabile a sinistra. A destra c'è `x + 3`: `x` vale 6, quindi $6 + 3 = 9$, e il 9 viene messo dentro `x`. La variabile viene **sovrascritta** con un nuovo valore.

Definizione

L'operazione `x = x + a`, dove `a` è un numero qualunque (o anche un'altra variabile), si chiama **incremento**: si prende la variabile e le si **aggiunge un valore**.

3 L'assegnazione composta: +=, -=, *=, /=

I programmatori sono **pigri**, e per una buona ragione: **meno si scrive**, più è bassa la **probabilità di sbagliare**. Non vuol dire che scrivendo poco non si sbagli mai, ma una riga lunga ha più occasioni di errore di una corta. Per questo il C ha una **forma più compatta** per l'incremento:

```
x = x + a;
// si puo' scrivere, in modo equivalente:
x += a;
```

Il simbolo `+=` vuol dire: **prendi `x`**, **aggiungi `a`** e **salva il risultato** di nuovo in `x`. Per esempio, con `x = 7;` e poi `x += 3;`, `x` vale **10**. Attenzione: si parla di incremento quando si aggiunge un valore **alla stessa variabile**; se si devono sommare **più valori diversi**, non è più un incremento, è una normale **somma**.

Nota

La `a` è solo un modo per indicare **un numero qualunque**: può essere 3, 4, 12, oppure una variabile. Per esempio, dopo `int a; a = 12;`, scrivere `x = x + 12;` oppure `x += 12;` dà **lo stesso risultato**.

La stessa forma vale anche per le altre operazioni:

Forma compatta	Equivale a	Esempio	Risultato
`x += 3;`	`x = x + 3;`	x vale 7	x diventa 10
`x -= 3;`	`x = x - 3;`	x vale 20	x diventa 17
`x *= 2;`	`x = x * 2;`	x vale 5	x diventa 10
`x /= 2;`	`x = x / 2;`	x vale 10	x diventa 5

Il `-=` è un **decremento**, ma fa parte della stessa **famiglia degli incrementi**. La regola generale: quando si deve scrivere `x = x` *operazione numero*, si può **portare l'operatore** davanti all'uguale ed **eliminare la seconda x**.

Nota

Esiste la stessa forma anche per gli operatori **bit a bit**, per esempio `x &= ...` al posto di `x = x & ...`: serve per il **mascheramento dei dati**, che vedremo quando faremo il **firmware** e lavoreremo sui singoli bit.

4 L'incremento unitario: x++ e x--

Solo per il **più 1** e il **meno 1** c'è una forma **ancora più corta**:

Forma lunga	Forma compatta	Forma unitaria
`x = x + 1;`	`x += 1;`	`x++;`
`x = x - 1;`	`x -= 1;`	`x--;`

Due più consecutivi incrementano di 1 la variabile, **due meno** la decrementano di 1. Per esempio `x = 7; x++;` porta `x` a **8**. Lo useremo tantissimo nei **cicli**.

Attenzione

Il `++` vale **solo per il più 1**. Per aggiungere 2 non esiste `x+++`: si scrive `x += 2` oppure `x = x + 2`.

5 x++ oppure ++x: è una questione di quando

Il ++ si può scrivere **dopo** la variabile (x++) oppure **prima** (++x). Da solo, su una riga, fa la stessa cosa: **incrementa di 1**. La differenza si vede quando il valore viene **anche usato**, per esempio in un'assegnazione. In aula abbiamo confrontato questi due codici:

```
int x;
x = 7;
int y;
y = x++;
```

```
int x;
x = 7;
int y;
y = ++x;
```

Definizione

- ++x (e --x): **incrementa PRIMA** di usare il valore.
- x++ (e x--): **incrementa DOPO** aver usato il valore.

Codice	Cosa succede	Risultato
y = x++;	prima copia 7 in y, poi incrementa x	y = 7, x = 8
y = ++x;	prima incrementa x a 8, poi lo copia in y	y = 8, x = 8

In tutti e due i casi **x alla fine vale 8**: cambia solo il **valore che finisce** in y.

Attenzione

In aula, la prima spiegazione aveva **invertito** i due casi; il docente l'ha **corretta subito** alla lavagna (Figura 1). Se negli appunti hai scritto che con y = x++; anche y vale 8, **correggi**: con x++ la y riceve il valore **vecchio**, cioè **7**.

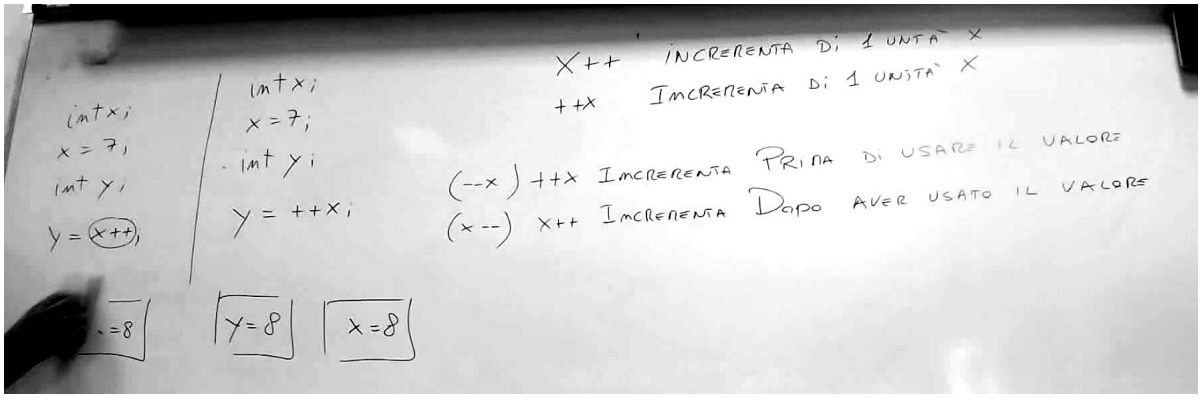


Figura 1: La lavagna con i due codici e la regola corretta: ++x incrementa prima di usare il valore, x++ incrementa dopo averlo usato. (Il riquadro sotto il primo codice è parzialmente coperto dalla mano.)

Nota

Una curiosità: polling e interrupt. Mentre aspettava che la classe finisse di copiare, il docente ha chiesto ogni tanto "posso andare avanti?". In informatica questa tecnica si chiama **polling**: si **chiede ripetutamente** a un dispositivo se ha finito. L'alternativa è l'**interrupt** (in italiano **interruzione**): è il dispositivo stesso ad **avvisare** quando ha finito.

6 Il costrutto switch

6.1 Il problema: troppi if

L'`if` sceglie quale codice eseguire in base a una **condizione**, che può essere solo **vera o falsa**. Se un programma chiede un numero da 1 a 10 e deve fare 10 cose diverse a seconda del numero, con l'`if` bisogna scrivere una lunga catena: `if (dato == 1) ...` `else if (dato == 2) ...` `else if (dato == 3) ...` e avanti così. Funziona, ma è **scomodo** e **confusionario**, e facilita gli **errori**.

Definizione

Lo **switch** è un costrutto che sceglie **che cosa fare** in base al **valore di una variabile**, in modo **ordinato**: è come un `if` con tanti `else`, ma con un'impaginazione molto più chiara e facile da gestire.

6.2 La struttura

```
switch (variabile_da_valutare)
{
    case opzione1:
        istruzione1;
        istruzione2;
        istruzione3;
        break;
    case opzione2:
        istruzione1;
        break;
    default:
        istruzione1;
        istruzione2;
}
```

- Tra le **parentesi tonde** dello `switch` va la **variabile da valutare**.
- Ogni **`case`** è un **valore possibile** della variabile, seguito dai **due punti**. `switch` e `case` sono le due **parole chiave** del costrutto.
- "istruzione" **non è un comando** del C: indica **qualunque sequenza di istruzioni**, una, due, cinquanta.
- Ogni `case` finisce con **`break;`**.
- Il **`default`** è l'ultima possibilità: viene eseguito quando la variabile **non corrisponde** a nessun `case`.

- I `case` possono essere **quanti si vuole**: 4, 7, 18, 25. Alla lavagna ce n'erano due solo per motivi di spazio.

6.3 L'esempio: un menu con tre scelte

```c
int scelta;
printf("Inserisci una scelta");
scanf("%d", &scelta);

switch (scelta)
{
    case 1:
        printf("Hai scelto la prima opzione");
        break;
    case 2:
        printf("Hai scelto la seconda opzione");
        break;
    case 3:
        printf("Hai fatto la terza scelta");
        break;
    default:
        printf("Non hai fatto una scelta valida");
}
```

Il programma legge un numero con `scanf`. Lo switch **confronta** la variabile `scelta` **con i casi**, uno alla volta: con 1, con 2, con 3. Quando trova un'**uguaglianza**, esegue il codice di **quel caso**. Se l'utente inserisce un valore diverso da 1, 2 o 3, viene eseguito il **`default`**: "Non hai fatto una scelta valida".

Nota

Il testo dentro i `printf` **lo sceglie il programmatore**: il terzo è scritto in modo diverso dagli altri apposta, per far vedere che non deve essere sempre uguale. In ogni `case` c'è una sola istruzione per motivi di spazio, ma potrebbero essercene **più di una**.

6.4 Il break: interrompe il costrutto

Definizione

Il **`break`** **interrompe l'esecuzione** del costrutto: quando la macchina lo incontra, **esce dallo switch** e prosegue con il codice che viene dopo la graffa di chiusura.

Se `scelta` vale 1, la macchina esegue `printf("Hai scelto la prima opzione");`, poi trova il `break` ed esce. Ma **cosa succede senza il break**? Dopo il primo `printf` la macchina **scende**: trovare `case 2:` **non le fa rifare** il confronto, va avanti ed esegue anche il `printf` del caso 2, e così via, **finché non trova un break** o non arriva in fondo.

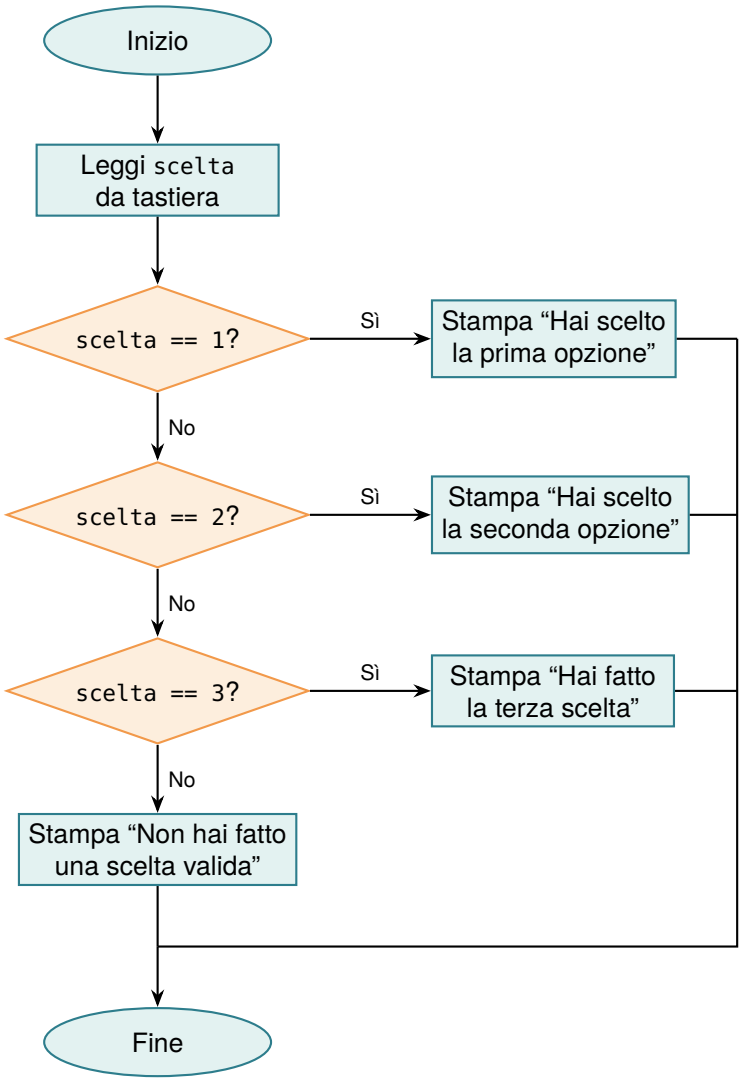


Figura 2: Il diagramma di flusso dello switch dell’esempio: i casi vengono confrontati uno alla volta; il `break` porta subito alla fine del costrutto.

Attenzione

Se i `break` **non si mettono**, lo switch **esegue tutto** quello che c’è sotto il caso trovato. Bisogna stare molto attenti: il **`break` va sempre messo**, tranne quando si vuole proprio questo comportamento (vedi sotto).

Esempio

Accorpare più scelte. Questo comportamento serve per fare la stessa cosa in più casi. Se voglio la stessa scritta quando `scelta` vale 1 **oppure** 2, scrivo i due `case` uno sotto l’altro:

```
case 1:
case 2:
    printf("Hai scelto o la prima o la seconda");
    break;
```

Con `scelta` uguale a 1 la macchina "scende" fino al `printf`; con `scelta` uguale a 2 ci arriva direttamente. Lo switch diventa così uno strumento molto potente per le **scelte multiple**.

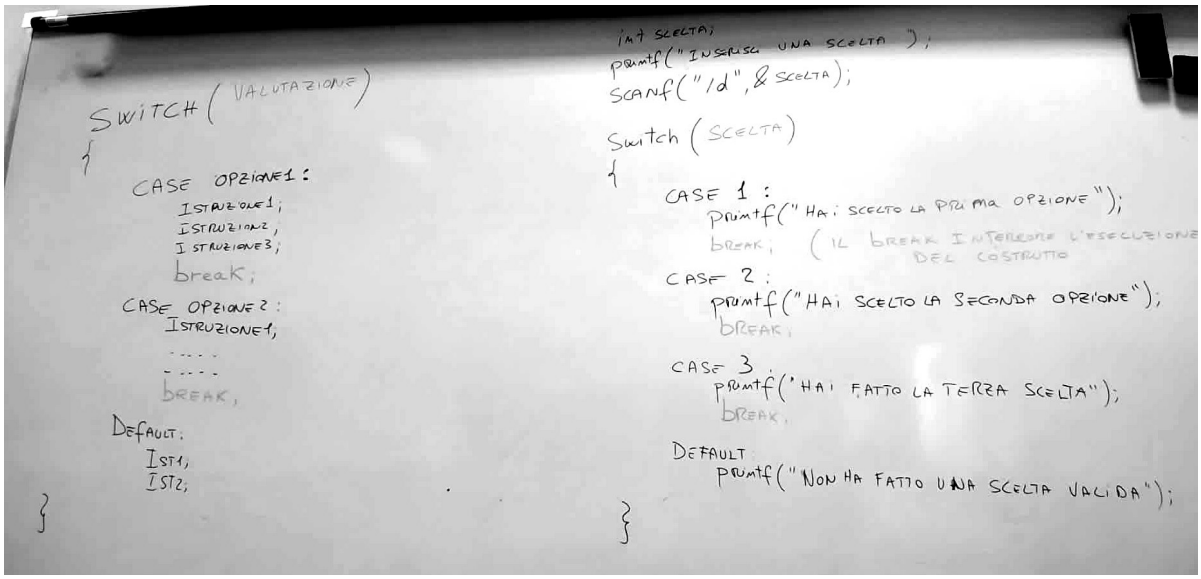


Figura 3: La lavagna: a sinistra la struttura generale dello switch, a destra l'esempio del menu con tre scelte e la nota "il break interrompe l'esecuzione del costrutto".

Nota

Tutti sì, oppure almeno un no. Per sapere se tutta la classe aveva finito di copiare, il docente ha fatto un esempio con la logica: "tutti hanno finito" è un **AND**: il primo dice sì **e** il secondo dice sì **e**... e l'ultimo dice sì. Il suo contrario, "non tutti hanno finito", è un **OR**: il primo dice no **oppure** il secondo dice no **oppure**... Passando da una frase al suo contrario, **AND diventa OR** e **ogni sì diventa no**: è il **teorema di De Morgan**. Lo ritroveremo nel `do while`.
E non si può scrivere semplicemente "tutti"? Si può scrivere in modo più intelligente, ma il controllo va fatto comunque: bisogna trovare il modo di **scrivere poco**, perché più si scrive più è probabile sbagliare. **La macchina non sbaglia**: sbaglia il **programmatore**.

Nota

Non mettete limiti all'utilità delle cose. Il nonno del docente si chiedeva a chi servisse il **telecomando**: oggi, se qualcuno ci chiedesse di alzarci per cambiare canale, ci sembrerebbe una follia. Anche le cose che sembrano più inutili possono rivelarsi **utilissime**.

7 I cicli di ripetizione

Il computer **non si annoia** a rifare sempre le stesse cose, ed è molto bravo a rifarle **sempre uguali**. I **cicli di ripetizione** servono a dirgli di ripetere un'operazione 10, 100, 1000 volte. Se devo far inserire 100 numeri, invece di scrivere 100 volte `printf` e `scanf`, scrivo un costrutto che **ripete** la stessa operazione 100 volte.

Definizione

In C i costrutti di ripetizione sono **tre**: **`for`**, **`while`** e **`do while`**. I primi due, pur avendo una struttura diversa, **fanno la stessa cosa**. Il terzo è una **variante del `while`**.

7.1 Il for

```
for (condizione di partenza; condizione di ripetizione; modificatore)
{
    istruzioni da ripetere;
}
```

Dentro le parentesi ci sono **tre parti**, separate dal **punto e virgola**:

1. la **condizione di partenza**: da dove si comincia a contare;
2. la **condizione di ripetizione**: fino a quando si continua;
3. il **modificatore**: come cambia il conteggio a ogni giro.

Attenzione

Le tre parti sono separate dal **punto e virgola**, **non** dalla virgola e **non** dai due punti: altrimenti il compilatore dà **errore**. È un errore facile: anche alla lavagna (Figura 5), nell'esempio dei 30 salti, al posto del secondo punto e virgola è scappata una virgola.

Esempio

I salti del volontario. In aula un allievo ha fatto 10 salti, contando ad alta voce. A parte saltare, che cosa ha fatto? Ha **tenuto il conto**: primo salto, secondo, terzo… e quando è arrivato a dieci **si è fermato**. E ha deciso da solo di **partire da uno**: nessuno gliel'aveva detto. Se avesse cominciato a contare da 90, o da 99 dovendo arrivare a 100, la regola "fermati a 100" sarebbe stata rispettata lo stesso, ma i salti sarebbero stati pochissimi: per questo il **punto di partenza** va deciso. Ecco le tre parti del `for`:

- un **punto di partenza** del conteggio, tenuto in una **variabile**;
- **fino a quando** ripetere: finché il conteggio è più piccolo del punto di arrivo, si continua a saltare;
- di quanto **incrementare** il conteggio a ogni salto: di uno.

Scriviamolo in C, con 30 salti e la variabile `s` (come "salti"):

```
for (int s = 0; s < 30; s++)
{
    salta;
}
```

- `int s = 0`: **crea** la variabile `s` e le dà il **valore di partenza**. Si parte da **zero**: gli informatici **contano da zero**, perché prima di cominciare non si è fatto nessun salto. La variabile si può anche creare prima del `for` (`int s;`) e scrivere solo `s = 0`.
- `s < 30`: si **ripete finché** `s` è minore di 30.
- `s++`: a ogni giro `s` **aumenta di 1**. Si poteva scrivere anche `s = s + 1` oppure `s += 1`; per saltare di 2 in 2, `s += 2`.
- `salta;` sta per **qualunque istruzione** da ripetere.

Il salto viene fatto con `s` che vale 0, 1, 2, 3, ... fino a 29: sono **30 salti**. Quando `s` diventa 30, la condizione `s < 30` è falsa e il ciclo **finisce**.

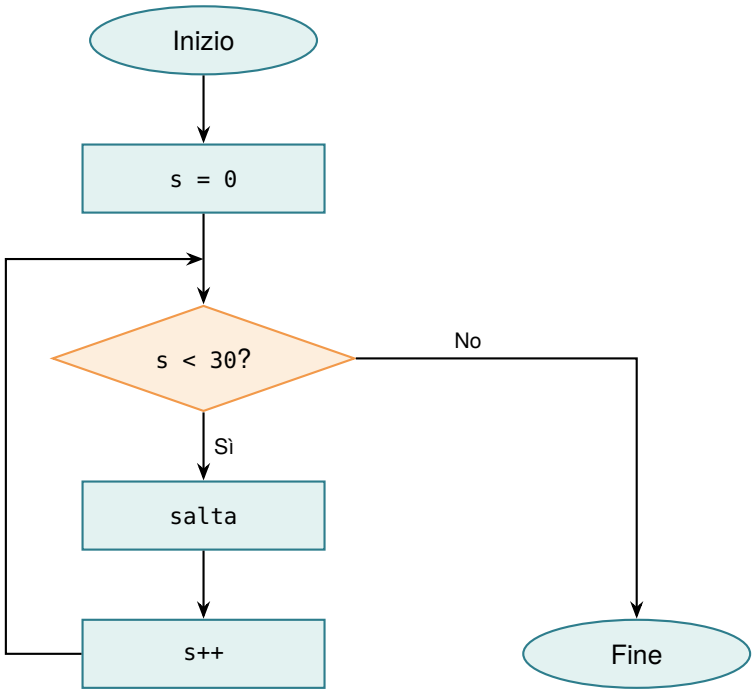


Figura 4: Il diagramma di flusso del ciclo dei 30 salti. È lo stesso per il `for` e per il `while` della Sezione 7.3: cambia solo il modo di scriverlo.

7.2 Attenzione a partenza e passo

Esempio

Quante volte salta questo ciclo?

```
for (s = 7; s < 30; s += 3)
{
    salta;
}
```

Molti guardano solo il 30 e rispondono "30 volte". **Non è vero**: il conteggio **parte da 7** e va avanti **di 3 in 3**.

s	7	10	13	16	19	22	25	28	31
s < 30?	sì	sì	sì	sì	sì	sì	sì	sì	**no**

Salta **8 volte**. È un esempio estremo, nella pratica si parte quasi sempre da 0, ma mostra che a decidere il numero di ripetizioni sono **tutte e tre le parti** insieme.

Attenzione

La condizione è la condizione per **continuare a ripetere**, **non** quella per fermarsi.

- `s < 30` e `s <= 29` sono **equivalenti** (il secondo ha solo un carattere in più, e il programmatore è pigro).
- `s == 29` è **sbagliato**: ripeterebbe solo quando `s` vale 29, quindi, partendo da 0, **mai**.

7.3 Il while

Il **`while`** fa la **stessa cosa** del `for`, scritta in modo diverso: la condizione di partenza **prima** del ciclo, la condizione di ripetizione **tra le parentesi**, il modificatore **dentro** il ciclo.

```
int s = 0;
while (s < 30)
{
    salta;
    s++;
}
```

"While" vuol dire **"mentre"**, "fintanto che": **fintanto che** `s` è minore di 30, ripeti. A ogni giro la macchina torna su e **si domanda** di nuovo se `s < 30`; quando `s` diventa 30, **esce**. Il modificatore `s++` si mette di solito **in fondo**, ma si può spostare a seconda di come si vuole che il ciclo si comporti. Il diagramma di flusso è quello della Figura 4.

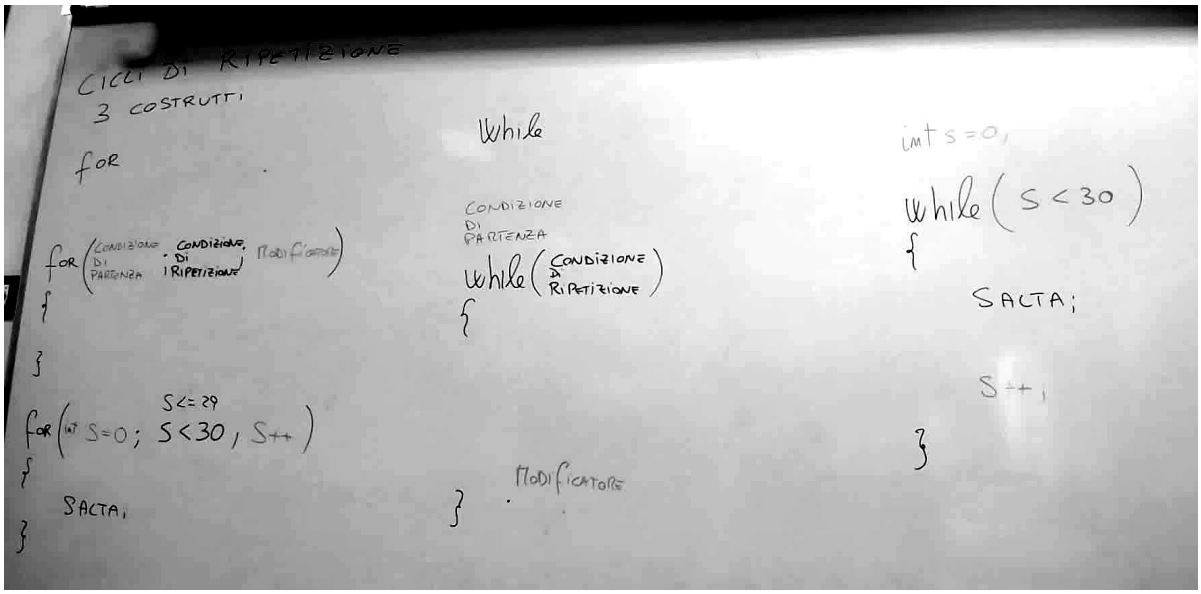


Figura 5: La lavagna: a sinistra il `for` (forma generale ed esempio dei 30 salti, con `s <= 29` come alternativa), al centro la forma generale del `while`, a destra l'esempio. Gli stessi colori indicano le parti che si corrispondono.

7.4 for o while? Una questione di stile

Si può usare **quello che si preferisce**: tutto quello che si fa con il `for` si può fare con il `while`, e viceversa. Ma c'è una **tendenza stilistica**:

Definizione

- **`for`**: quando si sa **a priori** quante volte ripetere, cioè lo **decide il programmatore** (fai 10 salti: sempre 10).
- **`while`**: quando **non si sa** quante volte ripetere, perché dipende da **quello che succede** mentre il programma gira, magari **all'esterno** del sistema (fai saltare finché non è stanco: quanti salti saranno, non si sa).

Esempio

- Far **lampeggiare** una lampadina **10 volte** per segnalare un errore: stilisticamente, **`for`**.
- Far lampeggiare una lampadina **finché** l'utente **non preme un pulsante**: **`while`**, perché non si sa quante volte lampeggerà.

Esempio

Combinare i due cicli. "Fai serie di 30 flessioni con una pausa tra una serie e l'altra, e vai avanti così finché non crolli": un **`while`** esterno ("continua finché non crolli") e, dentro, un **`for`** ("fai 30 flessioni"), più la pausa. Si possono combinare due `for`, due `while`, un `for` e un `while`: **tutto quello che serve**, una volta capite le tre parti e come configurarle.

Nota

Il docente ha programmato per 25 anni usando sempre solo il `for`; poi ha imparato a usare il `while` quando serve. Conoscere tutti e due è importante, perché un programmatore deve leggere anche il **codice degli altri**: in un gruppo di lavoro c'è sempre chi preferisce uno e chi l'altro.

7.5 Il do while

Il problema. Scriviamo un programma che chiede all'utente un **numero da 1 a 3**. Se l'utente inserisce un valore diverso (7, oppure -5), il programma **non lo accetta** e **ripete la domanda**, finché il numero non è giusto. Non si sa quante volte servirà: quindi un **`while`**. (Un `if` o uno `switch` potrebbero controllare se il numero è giusto, ma farebbero la domanda **una volta sola**.)

```
int x;
printf("Inserisci un numero da 1 a 3: ");
scanf("%d", &x);

while (x < 1 || x > 3)
{
    printf("Inserisci un numero da 1 a 3: ");
    scanf("%d", &x);
}
```

Qui il **modificatore** non è un `s++`: è la **nuova risposta** dell'utente, che cambia `x`. Il ciclo si ripete **finché** `x` è minore di 1 **oppure** maggiore di 3. Dopo il ciclo si può aggiungere un messaggio, per esempio "numero corretto": è una scelta del programmatore.

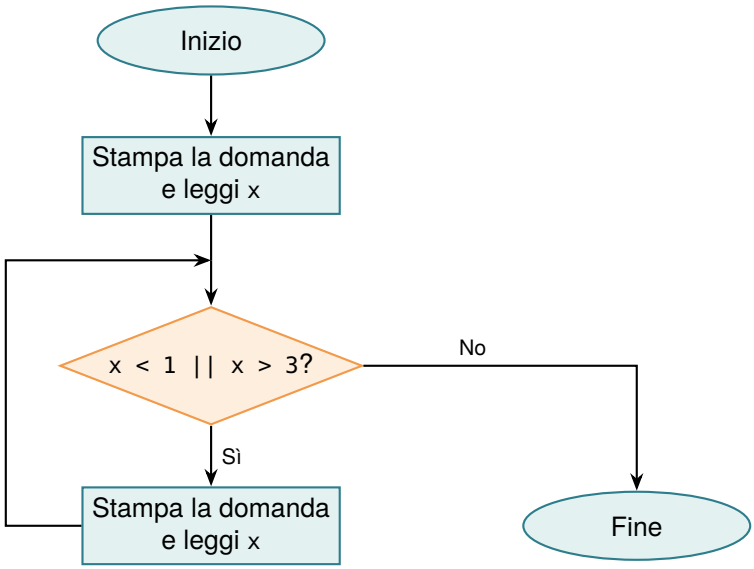


Figura 6: Con il `while` la domanda compare due volte: una prima del ciclo e una dentro.

Attenzione

Il problema di questa versione: `printf` e `scanf` sono **scritti due volte**. Il `while` controlla la condizione **prima** di entrare nel ciclo, quindi `x` deve avere già un valore: serve una prima richiesta **fuori dal ciclo**. Se la si togliesse e `x` contenesse per caso un numero valido (per esempio 2), il ciclo **verrebbe saltato** e la domanda **non verrebbe mai fatta**. Scrivere due volte la stessa cosa è brutto: è proprio il problema che risolve il `do while`.

```c
int x;
do
{
    printf("Inserisci un numero da 1 a 3: ");
    scanf("%d", &x);
} while (x < 1 || x > 3);
```

Definizione

Nel **`do while`** la condizione si controlla **in fondo**, dopo aver eseguito il blocco. Quindi il codice dentro il ciclo viene eseguito **almeno una volta**, anche se la condizione è falsa fin dall'inizio. Il `do while` si chiude con il **punto e virgola** dopo la parentesi del `while`.

Esempio

L'analogia dei soldi. Con il **`while`**, prima di andare a comprare qualcosa ti chiedo quanti soldi mi dai, e **prima controllo** se bastano: se sì, vado. Con il **`do while`** **ti do subito** 5 euro, e **poi** controlliamo se bastano per quello che devi comprare: se non bastano, si ricomincia.

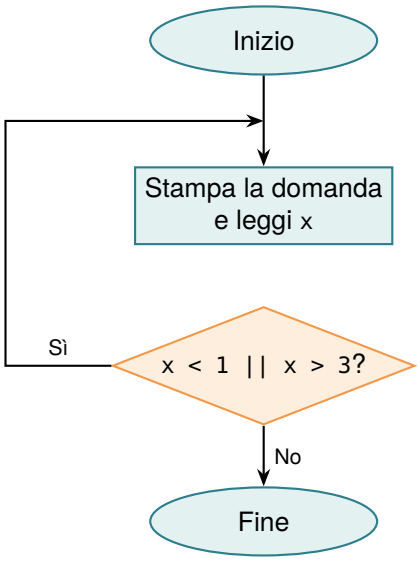


Figura 7: Con il `do while` la domanda è scritta una volta sola: prima si esegue, poi si controlla.

Attenzione

Condizione per ripetere e condizione "numero giusto". `x < 1 || x > 3` vuol dire "**il numero è sbagliato**": è la condizione per **ripetere** la domanda. La condizione "**il numero è giusto**" è `x >= 1 && x <= 3`, che è il suo **contrario**, non una forma equivalente: per il teorema di **De Morgan**, `x < 1 || x > 3` equivale a `!(x >= 1 && x <= 3)`. Nel ciclo va scritta la condizione per **continuare**: quindi `x < 1 || x > 3`, oppure `!(x >= 1 && x <= 3)`. Se in aula è stato annotato "`x >= 1 && x <= 3` è equivalente", va corretto così.

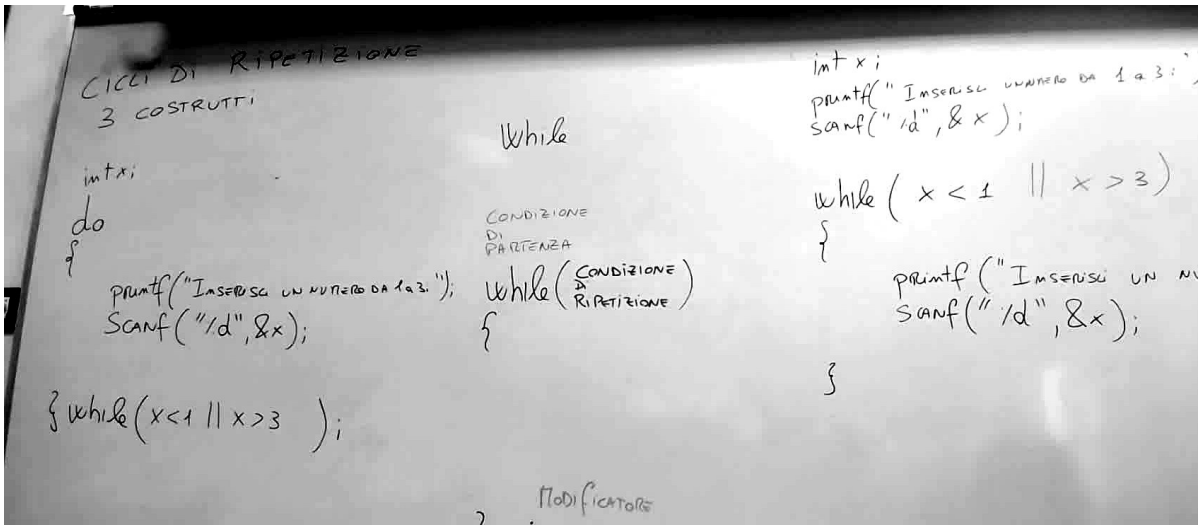


Figura 8: La lavagna: a sinistra il `do while`, a destra la versione con il `while`, con `printf` e `scanf` ripetuti prima e dentro il ciclo.

Definizione

Riassumendo: `for`, `while` e `do while` **ripetono** tutti e tre. `for` e `while` sono praticamente **interscambiabili**: decidono **prima** se ripetere. Il `do while` decide **dopo**: esegue il codice **almeno una volta**. Si usa meno spesso degli altri due, ma è giusto sapere che esiste.

8 Gli esempi provati su onlinegdb

`printf("*");` stampa un **asterisco**. Ripetuto, stampa asterischi **uno accanto all'altro**; con `\n` vanno **a capo**, uno sotto l'altro. In aula si è partiti scrivendo l'istruzione più volte:

```
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
printf("*\n");
```

Quanti asterischi? **8**, uno sotto l'altro. Scriverlo in maniera più intelligente vuol dire usare un **ciclo**:

```
#include <stdio.h>

int main()
{
    for (int s = 0; s < 180; s++)
    {
        printf("*");
    }
    return 0;
}
```

Stampa **180 asterischi** (il diagramma di flusso è quello della Figura 4, con 180 al posto di 30 e `printf("*")` al posto di `salta`). Poi lo stesso conteggio è stato riscritto con il `while`, con 30 ripetizioni (`s` da 0 a 29), e con il `for` equivalente: il risultato è **lo stesso**, 30 asterischi. Da qui in poi si è aggiunto `\n` nel `printf`, per andare a capo e vedere meglio. In aula si sono provate alcune varianti (qui scritte tutte con il `for`):

Ciclo	Valori di `s`	Asterischi
`for (s = 0; s < 15; s++)`	0, 1, 2, . . . , 14	15
`for (s = 0; s <= 15; s++)`	0, 1, 2, . . . , 15	16
`for (s = 0; s < 15; s += 2)`	0, 2, 4, 6, 8, 10, 12, 14 (la tabellina del 2)	8
`for (s = 2; s < 15; s += 3)`	2, 5, 8, 11, 14	5
`for (s = 0; s < 15; s += 15)`	0	1

8.1 Il ciclo infinito

Poi il `do while`, prima **senza modificatore**:

```
int s = 2;
do
{
    printf("*");
} while (s < 10);
```

Quante volte stampa? **Infinite volte**: da nessuna parte c'è scritto **come cambiare** `s`, quindi `s` resta 2 e la condizione è **sempre vera**. Se si torna fra dieci anni, il programma è ancora lì che gira. Mancava il modificatore: aggiungendo `s++;` dentro il ciclo, `s` va da 2 a 9 e gli asterischi sono **8** (Figura 9). Il diagramma di flusso è come quello del `do while` della sezione precedente: si esegue il blocco, poi si controlla `s < 10`.

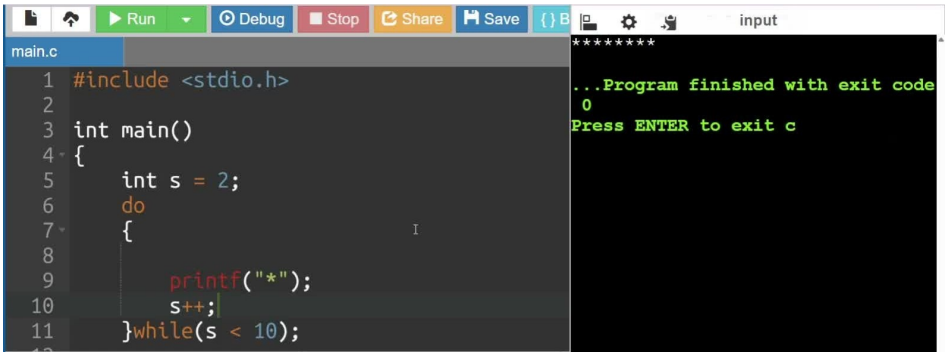


Figura 9: Il `do while` con il modificatore `s++`: partendo da 2 e finché `s < 10`, stampa 8 asterischi.

Nota

A volte il ciclo infinito si vuole. Il **computer di bordo** di un aereo esegue il suo programma da quando viene acceso a quando viene spento: se il programma arrivasse alla fine e si fermasse, l'aereo resterebbe senza controllo. Nei sistemi che **non devono mai fermarsi** si usa apposta un **`while` infinito**, in modo che quello che c'è dentro venga ripetuto per sempre.

9 Domande dalla classe

Nota

- **Il sorgente e il programma.** Il **sorgente** è il **codice**, quello che scriviamo noi per far accadere delle cose. Quello che la persona usa è il **programma eseguibile**. Il formato **`.exe`** è quello dei programmi di **Windows**: gli altri sistemi operativi avviano i programmi in altro modo. Molti programmi (per esempio la pagina di accesso della scuola) sono **prodotti commerciali**: chi li compra riceve il programma, ma **non il sorgente**, se il progetto non è open source.
- **Si programma anche in Linux?** Sì: queste basi valgono in **Linux**, **Windows** e **Mac**, e si ritrovano in **qualunque linguaggio**: C, Python, Rust… Cambia un po' la forma, ma i **concetti** sono gli stessi.

- **Che linguaggi usa il docente?** Soprattutto **C** e **C#**.
- **Si può colorare il testo?** La domanda è troppo ampia: dipende da dove, come e perché. Per un documento di Word c'è una funzione già pronta; per il testo stampato dai nostri programmi si può fare, ma dipende dallo strumento che si usa. Una domanda deve essere **specifica** per avere una risposta precisa.
- **E l'intelligenza artificiale?** Aiuta molto a fare le cose in fretta, ma il codice scritto da un'IA **non è perfetto**: bisogna sempre **rileggerlo** e metterci le mani.

10 Per esercitarsi

Esempio

Nella prossima lezione si fanno **esercizi** su `for`, `while` e `switch`. Chi vuole può provare già a casa, per esempio su onlinegdb: **non c'è nessuna consegna**, e se non riesce non importa, si fa insieme in aula. (Il docente non assegna mai un lavoro da un giorno per l'altro.) Gli esercizi qui sotto sono una **proposta della dispensa**, non assegnati in aula:

1. Stampare con un `for` 20 asterischi su una riga, poi 20 asterischi uno sotto l'altro.
2. Stampare la tabellina del 5 (5, 10, 15, ..., 50) con un `for`, poi con un `while`.
3. Un menu con `switch`: 1 = "Buongiorno", 2 = "Buonasera", 3 = "Buonanotte", altrimenti "Scelta non valida".
4. Chiedere un numero da 1 a 10 con un `do while`, ripetendo la domanda finché il numero non è valido.

11 Cosa ricordare di questa lezione

- `x = x + a` si può scrivere **`x += a`**; lo stesso per `-=`, `*=`, `/=`.
- `x++` e `x--` aggiungono o tolgono **1**. **`++x`** incrementa **prima** di usare il valore, **`x++`** **dopo**: con `x = 7`, `y = x++` dà `y = 7`, `y = ++x` dà `y = 8`; in entrambi i casi `x = 8`.
- Lo **switch** confronta una variabile con i **`case`**; il **`break`** interrompe il costrutto; senza `break` si **prosegue** nei casi successivi; **`default`** raccoglie tutti gli altri valori.
- Il **`for`** ha tre parti separate dal **punto e virgola**: partenza, condizione di ripetizione, modificatore. `for (int s = 0; s < 30; s++)` ripete **30 volte**.
- La condizione è quella per **continuare**, non per fermarsi.
- **`for`** quando il numero di ripetizioni è **deciso a priori**, **`while`** quando **dipende da quello che succede**.
- Il **`do while`** controlla la condizione **in fondo**: esegue il codice **almeno una volta**; finisce con **`;`**.
- Senza modificatore il ciclo è **infinito**: di solito è un errore, ma nei sistemi che non devono mai fermarsi si usa apposta.